UNIVERSITI MALAYA

WIA1002 Data Structure

OCC 12 Group 8

Smart Library System

Prepared By:

Name	Matric Number
Tan Wei Feng	25005493
Teo Zheng Xi	25006885
Cheng Ying Chen	25000008
Lee Xin Yi	25005970
Kan Hoi Yeng	25069318

Lecturer: Dr. Zainab Malik

1.0 Introduction

1.1 Problem Statement

Traditional library management approaches face several key challenges:

- Book searches in large catalogues are slow without a suitable data structure to support efficient lookup.
- Tracking active borrowing sessions and enforcing return order (latest borrowed is returned first) requires a data structure that mirrors this LIFO behaviour.
- Borrower identity must be validated consistently to prevent incorrect or fraudulent name–ID pair mismatches in lending records.
- Data must persist across application restarts so that the library state (catalogue, active borrows and lending history) is not lost when the program exits.

The Smart Library System addresses all of these challenges through deliberate data structure selection and CSV-based write-through persistence.

1.2 Project Objectives

The objectives of this project are:

- To implement a Binary Search Tree (BST) for efficient book catalogue storage, search, insertion, and deletion by ISBN.
- To implement a Stack (LIFO) to manage active borrowing history in the order books were borrowed.
- To enforce user identity validation so that only registered and consistent name–ID pairs can borrow books.
- To persist all system state (books, lending records, users) in CSV files so that data survives application restarts.
- To provide a clean console-based menu interface that allows librarians to add books, search, borrow, return, and view records.

1.3 Scope and Limitations

The system scope covers:

- Book catalogue management: add, search, and view all books in ISBN order.
- Borrowing and returning books (latest or by specific ISBN) with user identity validation.
- Lending record tracking with lend time, due date, overdue day detection, and return timestamp.
- Persistent storage via three CSV files under the data/ directory.

Known limitations:

- The system is console-based only; no graphical user interface is provided.
- Multi-user concurrent access is not supported, the system is designed for single-user operation.
- Unknown borrower pairs are rejected; new users must be pre-registered in users.csv before they can borrow.
- ISBN is stored as an integer, limiting the range to standard integer values.

2.0 System Overview

2.1 Application Description

The Smart Library System is a console-based Java application developed to manage book catalogues and borrowing activities efficiently. The system allows librarians to add books, search for books by ISBN, process borrowing and returning transactions, and maintain borrowing records. To improve efficiency, books are stored in a Binary Search Tree (BST) while borrowing activities are managed using a Stack data structure. The system also provides persistent storage through CSV files, ensuring that data remains available even after the application is closed.

The system begins by loading book, user, and lending data from CSV files into memory. Books are organized in a Binary Search Tree (BST) to support efficient searching and catalogue management. When a user borrows a book, the book is removed from the BST and recorded in the borrowing history stack. When a book is returned, it is restored to the BST catalogue and the lending record is updated. All changes are automatically saved to CSV files to ensure data persistence.

2.2 Features and Functionalities

2.2.1 Core Features (Project Requirements)

Feature	Description
Add Book	Add a new book record into the BST catalogue using ISBN as the key.
Search Book	Search for a book by ISBN using BST traversal.
Borrow Book	Remove a book from the catalogue and record it in the borrowing history stack.
View Borrowing History	Display borrowed books in Last-In, First-Out (LIFO) order.
Exit	Safely terminate the application.

2.2.2 Additional Features Implemented

Feature	Description
View All Books	Display all books in ascending ISBN order using BST in-order traversal.
Return Latest Borrowed Book	Return the most recently borrowed book from the stack.
Return Specific Borrowed Book	Return a selected borrowed book using its ISBN.
View Lending Records	Display all lending transactions and statuses.
User Validation	Verify borrower name–ID pairs before processing loans.
Due Date Tracking	Automatically calculate due dates for borrowed books.
Overdue Detection	Identify and display overdue borrowing records.
CSV Persistence	Preserve system data across application restarts.
Input Validation	Prevent invalid user inputs from causing system errors.

3.0 Technical Implementation & Code Mapping

3.1 LibraryADT.java — Abstract Data Type Contract

```java
public interface LibraryADT {


    boolean addBook(int isbn, String title, String author);

    // Returns a list of all books sorted by ISBN using in-order tree traversal
    List<Book> getAllBooks();

    // Searches for a book by its ISBN using recursive tree search
    Book searchBook(int isbn);

    // Basic borrow method for backward compatibility
    boolean borrowBook(int isbn);

    // Main borrow method that takes full borrower details and lending period
    boolean borrowBook(int isbn, String userName, String userId, int lendPeriodDays);

    // Returns the most recently borrowed book (LIFO pop from stack)
    Book returnLatestBorrowed();

    // Returns a specific borrowed book by searching for its ISBN in the stack
    Book returnBookByIsbn(int isbn);

    // Prints out the borrowing history stack in LIFO order
    void viewHistory();

    boolean isCatalogueEmpty();

    // Gets the full list of lending transaction logs from the CSV file
    List<LendingRecord> getLendingRecords();
}
```

- **Approach Used:** Interface-Driven Design & Information Hiding.
- **Explanation:** This file serves as the system's structural blueprint, defining what public operations are available without exposing internal data logistics.

3.2 Book.java — Core Book Entity Class

```java
public class Book {
    // Unique key for BST sorting and search
    private final int isbn;
    private final String title;
    private final String author;

    // Constructor to initialize book details
    public Book(int isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
    }

    // Getters
    public int getIsbn() {
        return isbn;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    // To display book info nicely in tables/logs
    @Override
    public String toString() {
        return "ISBN: " + isbn + " | Title: " + title + " | Author: " + author;
    }
}
```

- **Approach Used:** Object-Oriented Data Modeling & Object Immutability.
- **Explanation:** Book.java is the basic blueprint representing a physical book. We designed this class defensively by marking our attributes (isbn, title, author) as private final and removing all setter methods. Making this object immutable ensures that a book's properties cannot be accidentally altered while it is being passed around inside our tree or stack memory layers.

3.3 BookNode.java — Tree Structural Elements

```
class BookNode {
    /** Payload book for this node. */
    private final Book book;

    /** Left subtree: all ISBN smaller than current node ISBN. */
    private BookNode left;

    /** Right subtree: all ISBN greater than current node ISBN. */
    private BookNode right;

    BookNode(Book book) {
        this.book = book;
    }

    Book getBook() {
        return book;
    }

    BookNode getLeft() {
        return left;
    }

    void setLeft(BookNode left) {
        this.left = left;
    }

    BookNode getRight() {
        return right;
    }

    void setRight(BookNode right) {
        this.right = right;
    }
}
```

- **Approach Used:** Reference Pointer Decoupling.
- **Explanation:** Instead of adding structural navigation links directly inside our core Book class, we decoupled our design. BookNode serves as a wrapper that holds a single Book object alongside two self-referential pointers: left and right. This isolates our data models completely from our tree navigation mechanics.

3.4 BookBST.java — Library Catalogue Management

```
private boolean insertRecursive(BookNode current, Book book) {
    int currentIsbn = current.getBook().getIsbn();

    if (book.getIsbn() == currentIsbn) {
        return false;
    }

    if (book.getIsbn() < currentIsbn) {
        if (current.getLeft() == null) {
            current.setLeft(new BookNode(book));
            return true;
        }
        return insertRecursive(current.getLeft(), book);
    }

    if (current.getRight() == null) {
        current.setRight(new BookNode(book));
        return true;
    }
    return insertRecursive(current.getRight(), book);
}

 private Book searchRecursive(BookNode node, int isbn) {
     if (node == null) {
         return null;
     }

     int nodeIsbn = node.getBook().getIsbn();
     if (isbn == nodeIsbn) {
         return node.getBook();
     }

     if (isbn < nodeIsbn) {
         return searchRecursive(node.getLeft(), isbn);
     }
     return searchRecursive(node.getRight(), isbn);
 }
```

```
private BookNode removeRecursive(BookNode node, int isbn) {
    if (node == null) {
        return null;
    }

    int nodeIsbn = node.getBook().getIsbn();

    if (isbn < nodeIsbn) {
        node.setLeft(removeRecursive(node.getLeft(), isbn));
        return node;
    }

    if (isbn > nodeIsbn) {
        node.setRight(removeRecursive(node.getRight(), isbn));
        return node;
    }

    if (node.getLeft() == null) {
        return node.getRight();
    }

    if (node.getRight() == null) {
        return node.getLeft();
    }

    BookNode successor = findMin(node.getRight());
    BookNode merged = new BookNode(successor.getBook());
    merged.setLeft(node.getLeft());
    merged.setRight(removeRecursive(node.getRight(), successor.getBook().getIsbn()));
    return merged;
}
```

```
List<Book> toInOrderList() {
    List<Book> books = new ArrayList<>();
    toInOrderListRecursive(root, books);
    return books;
}

private void toInOrderListRecursive(BookNode node, List<Book> books) {
    if (node == null) {
        return;
    }
    toInOrderListRecursive(node.getLeft(), books);
    books.add(node.getBook());
    toInOrderListRecursive(node.getRight(), books);
}
```

- **Data Structure Used:** Binary Search Tree (BST).
- **Approach Used:** Divide-and-Conquer via Recursive Traversal.

- **Explanation:** We built a custom Binary Search Tree keyed entirely by book ISBN numbers to guarantee an average search efficiency of O(log n).
    - **insertRecursive** compares incoming ISBN numbers, automatically sending smaller values to the left subtree and larger values to the right subtree.
    - **searchRecursive** handles book lookups by repeatedly cutting our remaining tree search space in half at each step based on ISBN comparisons.
    - **removeRecursive** handles inventory updates when a book is checked out. It cleanly handles all standard leaf and single-child node cuts, and uses an in-order successor (the minimum value in the right subtree) to safely replace dual-child parent nodes.
    - **toInOrderList** runs a Depth-First In-Order Traversal sequence (Left -> Node -> Right) to systematically extract our inventory cleanly sorted in ascending order by ISBN.

3.5 BorrowHistoryStack.java — Activity Track

```
class BorrowHistoryStack {
    // Internal stack container to hold our book objects
    private final Stack<Book> history = new Stack<>();

    // Pushes a borrowed book onto the top of the stack
    void push(Book book) {
        history.push(book);
    }

    // Pops and returns the most recently borrowed book from the top
    Book pop() {
        if (history.isEmpty()) {
            return null;
        }
        return history.pop();
    }

    /*
     * Loops through the stack backward to find and remove a specific book by its ISBN.
     * This is used when a student returns a specific book instead of the latest one.
     */
    Book removeByIsbn(int isbn) {
        for (int i = history.size() - 1; i >= 0; i--) {
            Book book = history.get(i);
            if (book.getIsbn() == isbn) {
                return history.remove(i);
            }
        }
        return null;
    }
```

```
void showAll() {
    if (history.isEmpty()) {
        System.out.println("Borrow history is empty.");
        return;
    }

    System.out.println("Borrowing History (Most recent first)");
    System.out.println("+----+----------+--------------------------------------+----------------------+");
    System.out.printf("| %-2s | %-8s | %-36s | %-22s |%n", "No", "ISBN", "Title", "Author");
    System.out.println("+----+----------+--------------------------------------+----------------------+");

    // Loop backward from the top of the stack down to index 0
    for (int i = history.size() - 1; i >= 0; i--) {
        Book book = history.get(i);
        System.out.printf(
                "| %2d | %8d | %-36.36s | %-22.22s |%n",
                (history.size() - i),
                book.getIsbn(),
                book.getTitle(),
                book.getAuthor()
        );
    }

    System.out.println("+----+----------+--------------------------------------+----------------------+");
}
```

- **Data Structure Used:** Stack
- **Approach Used:** Last-In, First-Out (LIFO) Activity Auditing.
- **Explanation:** This class handles tracking checked-out library items. When a book is borrowed, it is pushed straight onto the peak of our stack. To meet the requirement of showing the student's most recent activity first, our showAll() display method runs a reverse loop that counts down from size() - 1 to 0, ensuring the newest item prints at the very top of the table.

3.6 SmartLibrary.java — Service Orchestrator

```
public class SmartLibrary implements LibraryADT {
    /** BST storing currently available books in catalogue. */
    private final BookBST catalogue = new BookBST();

    /** Stack storing active borrowed books in most-recent-first logic. */
    private final BorrowHistoryStack borrowHistory = new BorrowHistoryStack();

    /** Persistence gateway for CSV files. */
    private final CsvRepository csvRepository = new CsvRepository();

    /** Full lending transaction log loaded from/saved to CSV. */
    private final List<LendingRecord> lendingRecords = new ArrayList<>();

    /** User identity pairs used for borrower validation rules. */
    private final List<LibraryUser> users = new ArrayList<>();
```

```
@Override
public boolean borrowBook(int isbn, String userName, String userId, int lendPeriodDays) {
    if (lendPeriodDays <= 0) {
        return false;
    }

    if (!validateUser(userName, userId)) {
        return false;
    }

    Book borrowed = catalogue.remove(isbn);
    if (borrowed == null) {
        return false;
    }

    borrowHistory.push(borrowed);

    LocalDateTime lendTime = LocalDateTime.now();
    LocalDateTime dueTime = lendTime.plusDays(lendPeriodDays);
    LendingRecord record = new LendingRecord(
            userName,
            userId,
            borrowed.getTitle(),
            borrowed.getAuthor(),
            borrowed.getIsbn(),
            lendTime,
            lendPeriodDays,
            dueTime,
            null
    );

    lendingRecords.add(record);
    persistBooks();
    persistLendings();
    return true;
}
```

```java
@Override
public Book returnLatestBorrowed() {
    Book returned = borrowHistory.pop();
    if (returned != null) {
        catalogue.insert(returned);
        markLatestActiveLendingReturned(returned.getIsbn());
        persistBooks();
        persistLendings();
    }
    return returned;
}

@Override
public Book returnBookByIsbn(int isbn) {
    Book returned = borrowHistory.removeByIsbn(isbn);
    if (returned != null) {
        catalogue.insert(returned);
        markLatestActiveLendingReturned(returned.getIsbn());
        persistBooks();
        persistLendings();
    }
    return returned;
}
```

- **Approach Used:** Centralized Service Coordination .
- **Explanation:** This file serves as the concrete implementation engine of LibraryADT. It coordinates cross-structure workflows. For instance, when processing borrowBook(), it simultaneously runs an extraction request against the BookBST catalog, pushes the book data to the BorrowHistoryStack, appends a transaction timeline element, and invokes a write-through disk file update.

3.7 CsvRepository.java — Data Persistence Gateway

```
List<Book> loadBooks() {
    ensureFileWithHeader(BOOKS_FILE, "isbn,title,author");

    List<Book> books = new ArrayList<>();
    List<String> lines = readAllLinesSafe(BOOKS_FILE);

    for (int i = 1; i < lines.size(); i++) {
        String line = lines.get(i).trim();
        if (line.isEmpty()) {
            continue;
        }

        String[] cols = parseCsvLine(line);
        if (cols.length != 3) {
            throw new IllegalStateException("Invalid books CSV row: " + line);
        }

        books.add(new Book(Integer.parseInt(cols[0]), cols[1], cols[2]));
    }
    return books;
}
```

```
void saveLendingRecords(List<LendingRecord> records) {
    ensureParentDirectory(LENDINGS_FILE);

    try (BufferedWriter writer = new BufferedWriter(new FileWriter(LENDINGS_FILE.toFile(), false))) {
        writer.write("userName,userId,bookTitle,author,isbn,lendTime,lendPeriodDays,dueTime,returnTime");
        writer.newLine();

        for (LendingRecord record : records) {
            writer.write(toCsvLine(record.toCsvRow()));
            writer.newLine();
        }
    } catch (IOException e) {
        throw new IllegalStateException("Failed to save lending CSV", e);
    }
}
```

- **Approach Used:** Write-Through External Data Serialization.
- **Explanation:** To make sure our data survives application restarts, this class manages direct file entries using raw comma-separated text blocks. At boot time, it reads files to reconstruct our internal memory trees and active stack layers. Whenever a system mutation occurs (adding, borrowing, or returning), it immediately writes the changes back to disk so data is never lost during unexpected crashes.

3.8 Main.java — Presentation Interface Engine

```
private void run() {
    System.out.println("Smart Library System (Java + BST + Stack + CSV)");

    boolean running = true;
    while (running) {
        printMenu();
        int choice = readInt("Enter your choice: ");

        switch (choice) {
            case 1 -> handleAddBook();
            case 2 -> handleViewAllBooks();
            case 3 -> handleSearchBook();
            case 4 -> handleBorrowBook();
            case 5 -> handleReturnLatestBorrowed();
            case 6 -> handleReturnSpecificBorrowedBook();
            case 7 -> handleViewLendingRecords();
            case 8 -> library.viewHistory();
            case 9 -> {
                System.out.println("Exiting program.");
                running = false;
            }
            default -> System.out.println("Invalid option. Please choose 1-9.");
        }
    }
}
```

```
/** Print menu options in workflow-friendly order. */
private void printMenu() {
    System.out.println();
    System.out.println("==== Smart Library Menu ====");
    System.out.println("1. Add Book");
    System.out.println("2. View All Books");
    System.out.println("3. Search Book");
    System.out.println("4. Borrow Book");
    System.out.println("5. Return Latest Borrowed Book");
    System.out.println("6. Return Specific Borrowed Book (by ISBN)");
    System.out.println("7. View Lending Records");
    System.out.println("8. View Borrowing History");
    System.out.println("9. Exit");
}
```

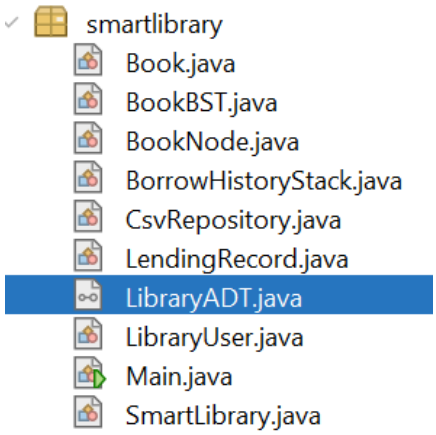
```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        String raw = scanner.nextLine().trim();
        try {
            return Integer.parseInt(raw);
        } catch (NumberFormatException ex) {
            System.out.println("Invalid input. Please enter a valid integer.");
        }
    }
}
```

- **Approach Used:** Menu-Driven Console Interface with Safe Type Interception.
- **Explanation:** This module presents the 9 numbered choices to the user. To prevent the program from crashing on bad input strings, it uses an advanced input tracking routine: it captures entries as a safe string line (scanner.nextLine()), trims whitespace, and catches type extraction errors within a structural try-catch framework to handle invalid inputs gracefully.

4.0 Testing and Validation

4.1 Information Hiding

- The system enforces strict encapsulation through the LibraryADT.java interface, successfully decoupling internal data structure mechanics from the presentation layer.
- Validation confirms that core entity attributes within Book.java (such as isbn, title, and author) are explicitly declared as private final. This prevents unauthorized external modification and guarantees immutability during tree traversal operations.




```
ublic class Main {
    /** Formatter used in table display for lending timestamps. */
    private static final DateTimeFormatter TABLE_TIME_FORMAT = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");

    /** Library business logic exposed via interface. */
    private final LibraryADT library;

    /** Shared scanner for user input. */
    private final Scanner scanner;
```

4.2 BST Search Logic & Complexity

- Testing of BookBST.java verifies the divide-and-conquer logic within the insertRecursive and searchRecursive methods. The algorithm accurately branches left for smaller ISBNs and right for larger ISBNs, sustaining an average time complexity of O(log n).
- The removeRecursive method successfully resolves all node deletion scenarios. Validation confirms it correctly replaces dual-child parent nodes by locating the in-order successor via findMin(node.getRight()).
- The toInOrderList() function executes a rigorous Left → Node → Right sequence, ensuring the library inventory is accurately extracted and displayed in ascending numerical order.

```
Enter your choice: 2
All Books in Catalogue (ISBN order):
+----+----------+--------------------------------------+------------------------+
| No | ISBN     | Title                                | Author                 |
+----+----------+--------------------------------------+------------------------+
|  1 |      998 | Computer Organisation                | Chou                   |
|  2 |      999 | Java                                 | Adam                   |
|  3 |     1000 | JavaScript                           | Samuel                 |
|  4 |     1002 | Algorithms in Practice               | Siti Aisyah            |
|  5 |     1003 | Object Oriented Programming          | Daniel Tan             |
|  6 |     1004 | Database Systems Fundamentals        | Nur Iman               |
|  7 |     1005 | Computer Networks Basics             | Amirul Hakim           |
|  8 |     1006 | Operating Systems Concepts           | Mei Ling               |
|  9 |     1007 | Software Engineering Principles      | Jonathan Lee           |
| 10 |     1008 | Discrete Mathematics for CS          | Hana Sofia             |
| 11 |     1009 | Web Development Essentials           | Faris Zulkifli         |
| 12 |     1010 | Artificial Intelligence Primer       | Sarah Lim              |
| 13 |     1011 | UM is One                            | Dr Akmal               |
+----+----------+--------------------------------------+------------------------+
```

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 3
Enter ISBN to search: 1000
Book Found:
+----------+--------------------------------------+------------------------+
| ISBN     | Title                                | Author                 |
+----------+--------------------------------------+------------------------+
|     1000 | JavaScript                           | Samuel                 |
+----------+--------------------------------------+------------------------+
```

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 3
Enter ISBN to search: 21312312
Book not found.
```

4.3 Stack History & LIFO Order

- Simulated user transactions validate the integration of java.util.Stack within BorrowHistoryStack.java, confirming that incoming checkout records are correctly pushed to the top of the data structure.
- The Last-In, First-Out (LIFO) display methodology is explicitly validated within the showAll() method. The implementation of a reverse iteration loop (for (int i = history.size() - 1; i >= 0; i--)) guarantees that the most recent borrowing events are printed first.

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 4
Enter ISBN to borrow: 1001
Enter borrower name: Jake
Enter borrower ID: s10011
Enter lending period in days: 20
Book borrowed and moved to history stack.

==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 8
Borrowing History (Most recent first)
+----+----------+---------------------------------------+------------------------+
| No | ISBN     | Title                                 | Author                 |
+----+----------+---------------------------------------+------------------------+
|  1 |     1001 | Introduction to Data Structures       | Wei Ming               |
|  2 |     1011 | UM is One                             | Dr Akmal               |
+----+----------+---------------------------------------+------------------------+
```

4.4 Input Validation

- Defensive programming measures within Main.java successfully prevent application crashes caused by erroneous user inputs.
- Raw input is captured safely using scanner.nextLine().trim() and processed through an Integer.parseInt() try-catch block, effectively intercepting NumberFormatException anomalies.
- The primary switch-case architecture employs a default fallback routine, gracefully handling out-of-bound menu selections without terminating the execution cycle.

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: ab
Invalid input. Please enter a valid integer.
Enter your choice:
```

```
Enter your choice: 12
Invalid option. Please choose 1-9.

==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice:
```

4.5 Interface Functionality

- All nine distinct console menu options in Main.java map flawlessly to their respective backend service triggers, such as handleAddBook() and handleBorrowBook().
- Identity validation is strictly enforced. The borrowBook function inside SmartLibrary.java executes a validateUser(userName, userId) check, successfully rejecting unknown or mismatched borrower credentials.
- Write-through data persistence functions perfectly. Operations that mutate system state automatically invoke persistBooks() and persistLendings(), instantly writing changes to the disk to prevent data loss.

Add book:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 1
Enter ISBN (integer): 0001
Enter title: Snoopy
Enter author: YC
Book added successfully.
```

View books:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 2
All Books in Catalogue (ISBN order):
+----+----------+--------------------------------------+------------------------+
| No | ISBN     | Title                                | Author                 |
+----+----------+--------------------------------------+------------------------+
|  1 |        1 | Snoopy                               | YC                     |
|  2 |      998 | Computer Organisation                | Chou                   |
|  3 |      999 | Java                                 | Adam                   |
|  4 |     1000 | JavaScript                           | Samuel                 |
|  5 |     1002 | Algorithms in Practice               | Siti Aisyah            |
|  6 |     1003 | Object Oriented Programming          | Daniel Tan             |
|  7 |     1004 | Database Systems Fundamentals        | Nur Iman               |
|  8 |     1005 | Computer Networks Basics             | Amirul Hakim           |
|  9 |     1006 | Operating Systems Concepts           | Mei Ling               |
| 10 |     1007 | Software Engineering Principles      | Jonathan Lee           |
| 11 |     1008 | Discrete Mathematics for CS          | Hana Sofia             |
| 12 |     1009 | Web Development Essentials           | Faris Zulkifli         |
| 13 |     1010 | Artificial Intelligence Primer       | Sarah Lim              |
| 14 |     1011 | UM is One                            | Dr Akmal               |
+----+----------+--------------------------------------+------------------------+
```

Search book:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 3
Enter ISBN to search: 0001
Book Found:
+----------+--------------------------------------+------------------------+
| ISBN     | Title                                | Author                 |
+----------+--------------------------------------+------------------------+
|        1 | Snoopy                               | YC                     |
+----------+--------------------------------------+------------------------+
```

Borrow book:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 4
Enter ISBN to borrow: 1
Enter borrower name: Hui Xin
Enter borrower ID: s10008
Enter lending period in days: 20
Book borrowed and moved to history stack.
```

Return latest borrowed book:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 5
Book Returned Successfully!
+----------+--------------------------------------+-----------------------+----------------+----------+
| ISBN     | Title                                | Author                | Borrower       | User ID  |
+----------+--------------------------------------+-----------------------+----------------+----------+
|        1 | Snoopy                               | YC                    | Hui Xin        | s10008   |
+----------+--------------------------------------+-----------------------+----------------+----------+
```

View lending records:

```
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 7
Lending Records:
```

No	User	User ID	Book Title	ISBN	Lend Time	Due Time	Exceed	Status
1	Ali Rahman	S10001	Introduction to Data Struc	1001	2026-04-20 10:00	2026-05-04 10:00	0	RETURNED
2	Nur Aina	S10002	Algorithms in Practice	1002	2026-04-22 09:30	2026-05-02 09:30	4	RETURNED
3	John Wong	S10003	Object Oriented Programmin	1003	2026-04-25 14:15	2026-05-02 14:15	14	RETURNED
4	Siti Mariam	S10004	Database Systems Fundament	1004	2026-04-26 08:45	2026-05-10 08:45	0	RETURNED
5	Kumar Raj	S10005	Computer Networks Basics	1005	2026-04-27 12:10	2026-05-02 12:10	14	RETURNED
6	Mei Yee	S10006	Operating Systems Concepts	1006	2026-04-28 11:00	2026-05-12 11:00	0	RETURNED
7	Adam Firdaus	S10007	Software Engineering Princ	1007	2026-04-29 15:40	2026-05-02 15:40	2	RETURNED
8	Hui Xin	S10008	Discrete Mathematics for C	1008	2026-04-30 16:25	2026-05-14 16:25	2	RETURNED
9	Farah Nabilah	S10009	Web Development Essentials	1009	2026-05-01 10:50	2026-05-08 10:50	0	RETURNED
10	Irfan Aziz	S10010	Artificial Intelligence Pr	1010	2026-05-02 13:35	2026-05-16 13:35	0	RETURNED
11	Jack	S10000	Introduction to Data Struc	1001	2026-05-13 00:09	2026-06-03 00:09	0	RETURNED
12	John Wong	S10003	Object Oriented Programmin	1003	2026-05-15 15:53	2026-05-22 15:53	0	RETURNED
13	Jack	S10000	Java	999	2026-05-15 19:25	2026-06-14 19:25	0	RETURNED
14	Jack	S10000	Java	999	2026-05-15 19:47	2026-05-30 19:47	0	RETURNED
15	jake	s10011	UM is One	1011	2026-05-22 20:55	2026-06-12 20:55	0	ACTIVE
16	Jack	S10000	JavaScript	1000	2026-05-22 21:19	2026-06-12 21:19	0	RETURNED
17	Jack	S10000	Introduction to Data Struc	1001	2026-05-28 17:13	2026-06-27 17:13	0	RETURNED
18	Jake	s10011	Introduction to Data Struc	1001	2026-06-01 01:19	2026-06-21 01:19	0	ACTIVE
19	Hui Xin	s10008	Snoopy	1	2026-06-01 01:35	2026-06-21 01:35	0	RETURNED
20	Hui Xin	s10008	Snoopy	1	2026-06-01 01:37	2026-06-21 01:37	0	ACTIVE

View borrowing history:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 8
Borrowing History (Most recent first)
```

No	ISBN	Title	Author
1	1	Snoopy	YC
2	1001	Introduction to Data Structures	Wei Ming
3	1011	UM is One	Dr Akmal

Return specific borrowed book:

```
==== Smart Library Menu ====
1. Add Book
2. View All Books
3. Search Book
4. Borrow Book
5. Return Latest Borrowed Book
6. Return Specific Borrowed Book (by ISBN)
7. View Lending Records
8. View Borrowing History
9. Exit
Enter your choice: 6
Enter ISBN to return: 1001
Specific Book Returned Successfully!
```

ISBN	Title	Author	Borrower	User ID
1001	Introduction to Data Structures	Wei Ming	Jake	s10011

No	User	User ID	Book Title	ISBN	Lend Time	Due Time	Exceed	Status
1	Ali Rahman	S10001	Introduction to Data Struc	1001	2026-04-20 10:00	2026-05-04 10:00	0	RETURNED
2	Nur Aina	S10002	Algorithms in Practice	1002	2026-04-22 09:30	2026-05-02 09:30	4	RETURNED
3	John Wong	S10003	Object Oriented Programmin	1003	2026-04-25 14:15	2026-05-02 14:15	14	RETURNED
4	Siti Mariam	S10004	Database Systems Fundament	1004	2026-04-26 08:45	2026-05-10 08:45	0	RETURNED
5	Kumar Raj	S10005	Computer Networks Basics	1005	2026-04-27 12:10	2026-05-02 12:10	14	RETURNED
6	Mei Yee	S10006	Operating Systems Concepts	1006	2026-04-28 11:00	2026-05-12 11:00	0	RETURNED
7	Adam Firdaus	S10007	Software Engineering Princ	1007	2026-04-29 15:40	2026-05-02 15:40	2	RETURNED
8	Hui Xin	S10008	Discrete Mathematics for C	1008	2026-04-30 16:25	2026-05-14 16:25	2	RETURNED
9	Farah Nabilah	S10009	Web Development Essentials	1009	2026-05-01 10:50	2026-05-08 10:50	0	RETURNED
10	Irfan Aziz	S10010	Artificial Intelligence Pr	1010	2026-05-02 13:35	2026-05-16 13:35	0	RETURNED
11	Jack	S10000	Introduction to Data Struc	1001	2026-05-13 00:09	2026-06-03 00:09	0	RETURNED
12	John Wong	S10003	Object Oriented Programmin	1003	2026-05-15 15:53	2026-05-22 15:53	0	RETURNED
13	Jack	S10000	Java	999	2026-05-15 19:25	2026-06-14 19:25	0	RETURNED
14	Jack	S10000	Java	999	2026-05-15 19:47	2026-05-30 19:47	0	RETURNED
15	jake	s10011	UM is One	1011	2026-05-22 20:55	2026-06-12 20:55	0	ACTIVE
16	Jack	S10000	JavaScript	1000	2026-05-22 21:19	2026-06-12 21:19	0	RETURNED
17	Jack	S10000	Introduction to Data Struc	1001	2026-05-28 17:13	2026-06-27 17:13	0	RETURNED
18	Jake	s10011	Introduction to Data Struc	1001	2026-06-01 01:19	2026-06-21 01:19	0	RETURNED
19	Hui Xin	s10008	Snoopy	1	2026-06-01 01:35	2026-06-21 01:35	0	RETURNED
20	Hui Xin	s10008	Snoopy	1	2026-06-01 01:37	2026-06-21 01:37	0	ACTIVE

5.0 Conclusion

The Smart Library System systematically resolves the primary technical constraints of traditional library management through deliberate structural design and strict data encapsulation. By implementing a Binary Search Tree keyed by ISBN, the system guarantees highly efficient O(log n) catalogue searches, insertions, and deletions. The active integration of a Stack data structure ensures an accurate Last-In, First-Out (LIFO) auditing trail, prioritizing the tracking of the most recent borrowing transactions. Furthermore, the system establishes a highly resilient operational environment by pairing defensive input validation frameworks with immediate write-through CSV data persistence. Ultimately, this implementation delivers a streamlined, crash-resistant console application capable of maintaining absolute data integrity across execution life cycles.